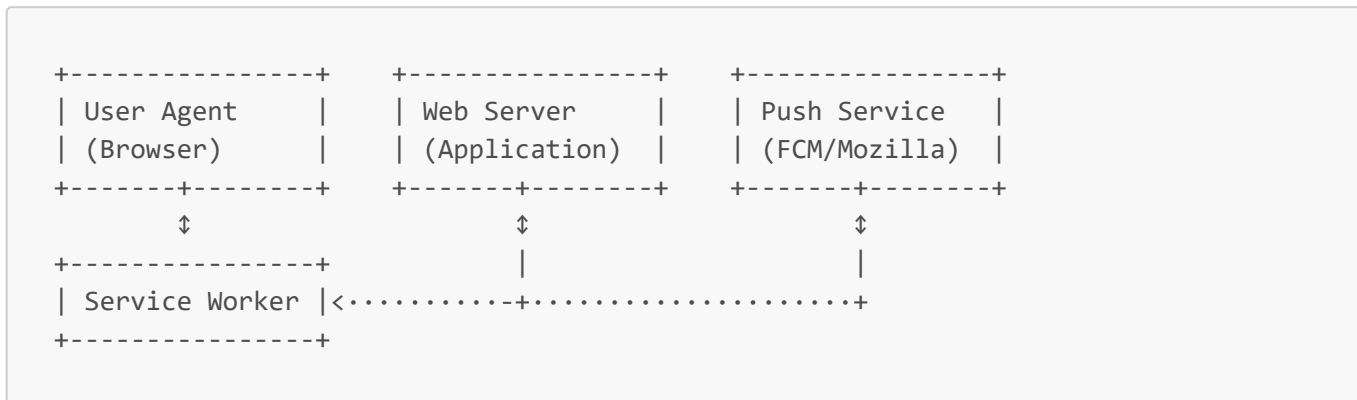


웹 푸시 알림 시스템 상세 흐름도

1. 시스템 구성요소 및 역할

A. 기본 아키텍처



B. 구성요소별 핵심 기능

1. User Agent (Browser)

- 알림 권한 관리
- Service Worker 등록 및 관리
- 구독 프로세스 처리
- 키 쌍(p256dh/auth) 생성

2. Service Worker

- 백그라운드에서 독립적으로 실행되는 JavaScript 코드
- 푸시 이벤트 처리 및 알림 표시
- 오프라인 기능 지원을 위한 캐싱
- 사용자 상호작용(알림 클릭) 처리

3. Web Server

- VAPID 키 관리
- JWT 토큰 생성 및 서명
- 메시지 암호화
- 구독 정보 관리

4. Push Service

- 메시지 중계
- VAPID 인증 검증
- 구독 엔드포인트 제공
- 메시지 전달 보장

C. 통신 흐름

- 실선(—): 직접 통신

- 점선(...): 간접 통신
- 양방향 화살표(↕): 양방향 데이터 흐름

2. Service Worker 생명주기 및 관리

A. 등록 및 초기화 프로세스

```
// 브라우저에서 Service Worker 등록
if ('serviceWorker' in navigator) {
  try {
    // 1. 브라우저가 sw.js 파일을 웹서버에서 다운로드하여 Service Worker 저장소에 저장
    const registration = await navigator.serviceWorker.register('/sw.js', {
      scope: '/' // Service Worker가 제어할 범위
    });

    // 2. 자동 업데이트 관리 설정
    setupAutoUpdate(registration);
  } catch (error) {
    console.error('Service Worker 등록 실패:', error);
  }
}
```

B. 생명주기 단계

1. 등록 (Registration)

- 브라우저가 Service Worker 스크립트를 다운로드, 파싱, 실행
 - 브라우저는 navigator.serviceWorker.register('/sw.js') 호출 시 스크립트를 다운로드하고 파싱한 후 실행합니다.
- 별도의 스레드에서 독립적으로 실행. 각각의 origin 도메인마다 스레드가 형성된다.
 - Service Worker는 메인 JavaScript 스레드와 분리된 별도의 워커 스레드에서 실행됩니다.
 - 각 origin(출처, 즉 프로토콜+도메인+포트)마다 독립적인 Service Worker 스레드가 생성됩니다.
 - 이는 서로 다른 웹사이트의 Service Worker가 서로 격리되어 실행됨을 의미합니다.
- HTTPS 환경에서만 동작 (localhost 제외)
 - 보안상의 이유로 Service Worker는 HTTPS 환경에서만 동작합니다.
 - 개발 편의를 위해 localhost(또는 127.0.0.1)에서는 예외적으로 HTTP에서도 동작합니다.

2. 설치 (Installation)

```
self.addEventListener('install', event => {
  event.waitUntil(
    Promise.all([
      cacheAssets(), // 필요한 리소스 캐싱
      self.skipWaiting() // 대기 단계 생략 (선택사항)
    ])
  );
});
```

◦ 쉬운 설명:

- 설치 단계는 Service Worker가 브라우저에 처음 다운로드되어 실행되는 시점입니다
- 마치 새 앱을 휴대폰에 설치하는 것과 비슷합니다
- 이 단계에서는 주로 앱이 오프라인에서도 작동하는 데 필요한 파일들을 미리 저장(캐싱)합니다
- `cacheAssets()`는 HTML, CSS, JavaScript, 이미지 등 필요한 파일을 미리 저장하는 함수입니다
- `self.skipWaiting()`은 "바로 설치 완료하기" 버튼과 같아서, 대기 없이 바로 활성화 단계로 넘어갑니다

◦ 대기 상태란:

- 이미 다른 Service Worker가 작동 중일 때, 새 Service Worker는 기본적으로 대기 상태에 들어갑니다
- 마치 새 버전의 앱이 설치되었지만 아직 활성화되지 않은 상태와 같습니다
- 사용자가 모든 탭을 닫고 다시 열 때까지, 또는 `skipWaiting()`을 호출할 때까지 대기합니다

3. 활성화 (Activation)

```
self.addEventListener('activate', event => {
  event.waitUntil(
    Promise.all([
      cleanupOldCaches(), // 이전 캐시 정리
      self.clients.claim() // 페이지 제어권 획득
    ])
  );
});
```

◦ 쉬운 설명:

- 활성화 단계는 Service Worker가 실제로 작동을 시작하는 단계입니다
- 앱 설치 후 처음 실행하는 것과 비슷합니다
- 이 단계에서는 주로 이전 버전의 데이터를 정리합니다
- `cleanupOldCaches()`는 이전 Service Worker가 저장한 오래된 파일들을 삭제합니다
- `self.clients.claim()`은 "지금 바로 제어권 가져오기" 버튼으로, 현재 열려있는 모든 페이지의 제어권을 즉시 획득합니다

Service Worker의 `clients.claim()` 쉽게 이해하기

`self.clients.claim()`은 "현재 열려있는 모든 페이지의 제어권을 즉시 획득한다"는 의미인데, 이를 일상 생활에 빗대어 설명하겠습니다.

일상 예시로 이해하기

🏫 학교의 교장 선생님 교체

1. 기존 상황:

- 학교에는 현재 교장 선생님(기존 Service Worker)이 있어 학교 운영(페이지 제어)을 담당하고 있습니다.
- 학생들(열린 페이지들)은 이 교장 선생님의 지시에 따라 행동합니다.

2. 새 교장 부임:

- 새 교장 선생님(새 Service Worker)이 부임했습니다.
- 기본적으로, 새 교장은 모든 학생들이 하교한 후 다음날(브라우저 재시작) 출근했을 때부터 지시를 내릴 수 있습니다.

3. `clients.claim()` 사용:

- 새 교장이 "지금부터 내가 지시한다!"라고 선언합니다.
- 학생들은 하교하고 다음날 등교할 때까지 기다리지 않고, 즉시 새 교장의 지시를 따르기 시작합니다.

기술적 설명

Service Worker 활성화 시 `clients.claim()`을 호출하면:

1. 즉각적인 제어권 획득:

- 새 Service Worker가 같은 스코프 내의 모든 열린 페이지에 대한 제어권을 즉시 획득합니다.
- 기본적으로는 페이지를 새로고침하거나 브라우저를 재시작해야 새 Service Worker가 제어권을 얻습니다.

2. 사용자 경험 개선:

- 사용자가 페이지를 새로고침하지 않아도 새로운 기능이나 수정사항이 즉시 적용됩니다.
- 예를 들어, 새로운 오프라인 기능이나 캐싱 전략이 즉시 활성화됩니다.

실제 코드에서의 의미

```
self.addEventListener('activate', event => {
  event.waitUntil(
    Promise.all([
      cleanupOldCaches(),    // 이전 버전의 캐시 정리
      self.clients.claim()   // 열린 페이지들의 제어권 즉시 획득
    ])
  );
});
```

이 코드는 "새 버전이 설치되면, 오래된 파일들을 정리하고, 지금 사용 중인 모든 페이지에 즉시 새 버전을 적용해 주세요"라는 의미입니다.

결론

`self.clients.claim()`은 마치 새 관리자가 취임하자마자 "지금부터 내 방식으로 운영합니다"라고 선언하는 것과 같습니다. 이를 통해 사용자는 페이지를 새로고침하지 않아도 새로운 Service Worker의 기능을 즉시 사용할 수 있게 됩니다.

C. 이벤트 기반 업데이트 관리

```
// 이벤트 기반 업데이트 체크
['fetch', 'push', 'sync'].forEach(eventType => {
  self.addEventListener(eventType, () => {
    // 1시간 간격으로 제한하여 업데이트 체크
    if (Date.now() - lastUpdateCheck > 60 * 60 * 1000) {
      checkForUpdates(registration);
    }
  });
});
```

쉬운 설명:

이 부분은 "앱 자동 업데이트 확인" 설정과 비슷합니다. Service Worker가 최신 버전인지 주기적으로 확인하는 방법을 정의합니다.

1. 업데이트 확인 트리거:

- fetch - 사용자가 페이지를 로드하거나 데이터를 요청할 때
- push - 알림이 도착할 때
- sync - 오프라인 상태에서 저장한 작업을 온라인 상태에서 처리할 때

2. 업데이트 확인 제한 (쓰로틀링):

- 모든 이벤트마다 업데이트를 확인하면 서버에 너무 많은 요청을 보낼 수 있습니다
- 그래서 마지막 확인 시간으로부터 1시간이 지났을 때만 업데이트를 확인합니다
- 이것은 마치 "앱 업데이트 확인은 하루에 한 번만" 같은 설정입니다

3. 업데이트 전략:

- 자동: 하루에 한 번 또는 사용자가 앱을 사용할 때 새 버전이 있는지 확인
- 수동: 사용자가 "업데이트 확인" 버튼을 눌렀을 때만 확인
- 쓰로틀링: 업데이트 확인을 너무 자주 하지 않도록 제한 (예: 최소 1시간 간격)

쉽게 말해, 이 코드는 Service Worker가 새 버전이 있는지 확인하는 빈도를 조절하여, 서버에 부담을 주지 않으면서도 적절한 시점에 최신 버전을 제공하기 위한 것입니다.

Service Worker 업데이트 확인의 필요성

Service Worker가 새 버전이 있는지 확인해야 하는 이유는 다음과 같습니다:

1. 기능 업데이트 적용

- **새로운 기능 제공:** 웹 앱에 새로운 기능이 추가되면 Service Worker 코드도 업데이트되어야 합니다.

- **버그 수정:** 기존 Service Worker에 버그가 있을 경우, 이를 수정한 새 버전을 사용자에게 제공해야 합니다.
- **성능 향상:** 캐싱 전략이나 메시지 처리 방식이 개선된 새 버전을 적용하여 성능을 향상시킬 수 있습니다.

2. 보안 강화

- **보안 취약점 패치:** 보안 이슈가 발견되면 빠르게 새 버전을 배포하고 사용자 기기에 적용해야 합니다.
- **보안 정책 변경:** 웹 앱의 보안 정책이 변경되면 Service Worker도 이에 맞게 업데이트되어야 합니다.

3. 사용자 경험 개선

- **오래된 코드 문제:** 업데이트되지 않은 Service Worker는 새로운 API나 콘텐츠와 충돌할 수 있습니다.
- **일관된 경험:** 모든 사용자가 최신 버전의 Service Worker를 사용하도록 하여 일관된 사용자 경험을 제공합니다.
- **캐시 관리:** 새로운 리소스를 캐시하고 오래된 캐시를 정리해야 웹 앱이 올바르게 작동합니다.

4. 기술적 필요성

- **브라우저 캐싱 메커니즘:** Service Worker 파일은 브라우저에 의해 강력하게 캐싱되므로, 변경사항을 감지하고 적용하는 메커니즘이 필요합니다.
- **바이트 단위 비교:** 브라우저는 Service Worker 파일이 바이트 단위로 변경되었을 때만 새 버전으로 인식합니다.
- **즉시 적용 어려움:** 기본적으로 Service Worker 업데이트는 사용자가 모든 탭을 닫고 다시 열 때까지 적용되지 않으므로, 업데이트 확인 및 관리 로직이 필요합니다.

이러한 이유로 Service Worker는 정기적으로 업데이트를 확인하고, 새 버전이 있을 경우 적절한 전략(즉시 적용 또는 다음 방문 시 적용)에 따라 업데이트를 관리해야 합니다.

Service Worker의 캐싱 동작 원리

HTML, CSS, JavaScript, 이미지 등 필요한 파일을이라는 문맥은 Service Worker의 설치 단계에서 진행되는 파일 캐싱에 관한 설명입니다.

캐싱되는 파일의 의미

이 부분은 웹 애플리케이션이 오프라인에서도 작동하거나 네트워크 연결이 불안정할 때 빠르게 로드되도록 하기 위해 Service Worker가 브라우저의 캐시 스토리지에 미리 저장해 두는 파일들을 의미합니다.

파일 캐싱 과정

1. 대상 파일:

- 웹 서버에 있는 웹 애플리케이션의 핵심 파일들(HTML, CSS, JavaScript, 이미지 등)

2. 저장 위치:

- 브라우저의 Cache Storage API를 통해 관리되는 특별한 저장소에 저장됨
- 이 저장소는 일반적인 브라우저 캐시와는 별도로 Service Worker가 직접 제어

3. 코드 예시:

```
// Service Worker 설치 시 파일 캐싱 예시
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open('app-shell-v1').then(cache => {
      return cache.addAll([
        '/',
        '/index.html',
        '/styles/main.css',
        '/scripts/app.js',
        '/images/logo.png',
        // 기타 필요한 정적 자원들
      ]);
    })
  );
});
```

캐싱의 이점

- **오프라인 작동:** 인터넷 연결 없이도 웹앱 사용 가능
- **성능 향상:** 네트워크 요청 대신 캐시에서 빠르게 로드
- **네트워크 비용 절약:** 반복적인 다운로드 방지
- **안정성 향상:** 불안정한 네트워크 환경에서도 일관된 경험 제공

Service Worker는 이렇게 캐싱한 파일들을 관리하고, 네트워크 요청이 발생할 때 캐시된 버전을 제공할지 또는 네트워크에서 새로운 버전을 가져올지 결정하는 중간자 역할을 합니다.

3. VAPID 키 생성 및 설정

```
+-----+      +-----+      +-----+
| Web Server | --> | VAPID Keys | --> | Web Application |
| (Application) |   | - Public Key |   | (Client-side) |
+-----+      | - Private Key |   +-----+
                +-----+
```

- **용도:** 애플리케이션 서버가 푸시 서비스에 자신을 인증하고 신원을 증명
- **알고리즘:** ECDSA(Elliptic Curve Digital Signature Algorithm) P-256 사용
- **비공개키:** 서버에 안전하게 보관, JWT 토큰 서명에 사용
- **공개키:** 구독 과정에서 `applicationServerKey`로 전달

4. 구독 프로세스 상세 흐름

```
+-----+      +-----+      +-----+      +-----+
-----+
| Browser UI |   | User Agent |   | Web Server |   | Push
Service |
| (Permission Pop-up) | | (Browser JS) |   | (Application) |   |
```



단계별 설명

이 다이어그램은 다음과 같은 중요한 세부 사항을 명확히 표현하고 있습니다:

- 키 생성 주체:** 브라우저(User Agent)가 내부적으로 p256dh/auth 키 쌍을 생성합니다.
- 통신 흐름:** 브라우저는 푸시 서비스에 구독 요청을 보내고, 푸시 서비스는 endpoint URL만 반환합니다.
- PushSubscription 객체 구성:** 브라우저가 받은 endpoint URL과 자체 생성한 키를 결합하여 PushSubscription 객체를 구성합니다.
- 서버 전송:** 완성된 PushSubscription 객체가 웹 서버로 전송되어 저장됩니다.

PushSubscription 객체와 구독 정보

<설명> 4. PushSubscription 객체 반환

PushSubscription 객체에는 다음 정보가 들어 있습니다:

1. endpoint:

- 푸시 서비스(FCM/Mozilla 등)가 제공하는 고유 URL
- 이 URL로 메시지를 전송하면 특정 기기/브라우저로 전달됨

- 예: <https://fcm.googleapis.com/fcm/send/dxVEn-ACR8k:APA91bE...>

2. **expirationTime**:

- 구독 만료 시간 (옵션)
- **null**이면 만료되지 않음을 의미
- 특정 시간이 설정되어 있으면 해당 시간 이후에는 갱신 필요

3. **keys**: 메시지 암호화에 필요한 키 정보

- **p256dh**: ECDH P-256 공개키 (메시지 암호화에 사용)
- **auth**: 인증 비밀 (메시지 암호화 및 인증에 사용)

```
// PushSubscription 객체 예시
{
  "endpoint": "https://fcm.googleapis.com/fcm/send/dxVEn-ACR8k:APA91bE...",
  "expirationTime": null,
  "keys": {
    "p256dh": "BLGMvz2kBSL4riRwfc4XboJD-Dn9r7aZ9TdmhWA...",
    "auth": "tBHitJI4PYR9AOjDDk47Cw=="
  }
}
```

<설명> 5. 구독 정보를 서버로 전송

서버로 전송되는 구독 정보에는 위의 PushSubscription 객체와 함께 추가적인 정보가 포함될 수 있습니다:

1. 기본 PushSubscription 객체 전체:

- endpoint URL
- 암호화 키(p256dh, auth)
- 만료 시간 정보

2. 사용자 식별 정보 (애플리케이션에 따라 다름):

- 사용자 ID 또는 식별자
- 세션 정보 또는 기기 식별자

3. 추가 메타데이터 (선택적):

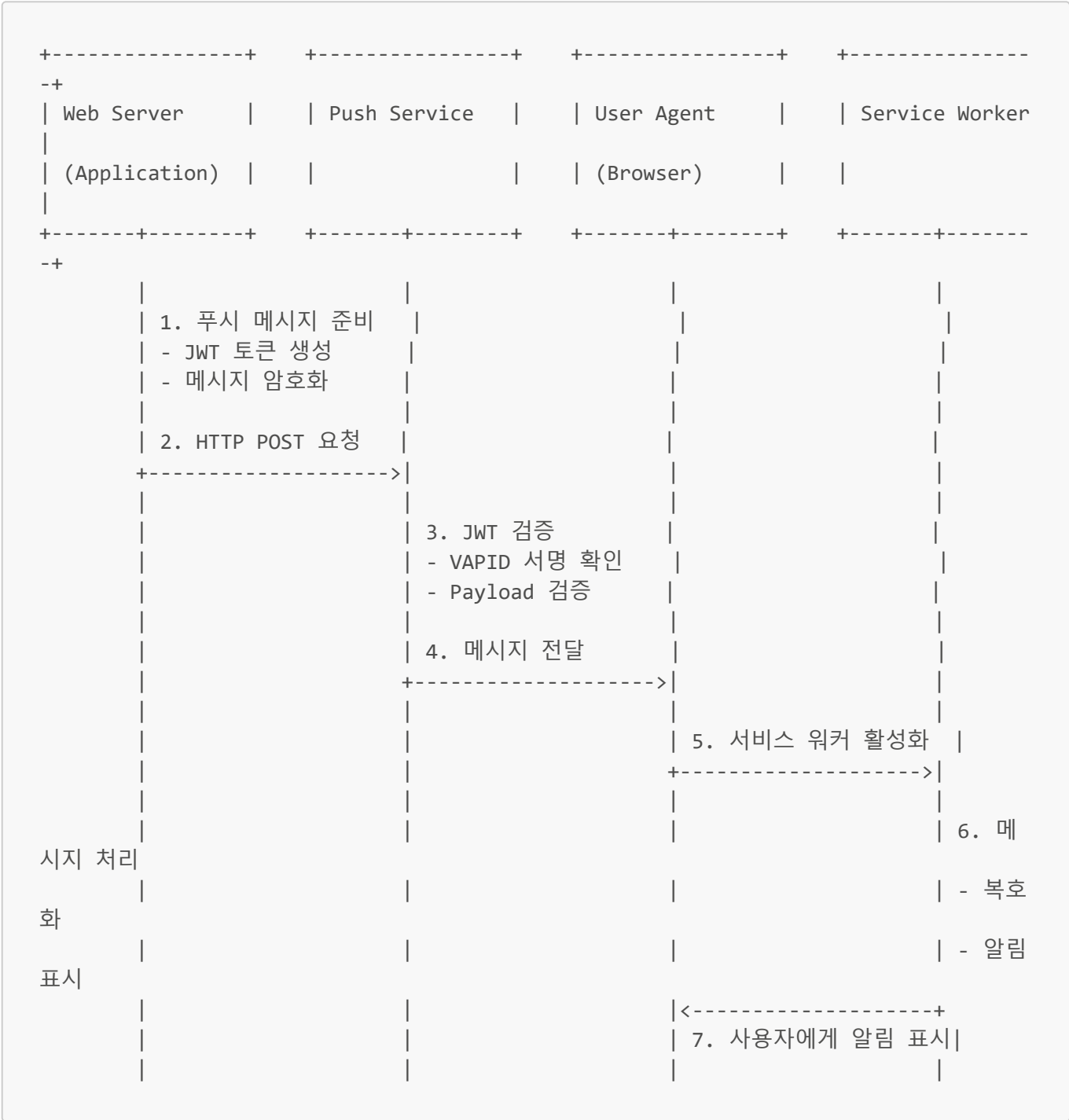
- 구독 생성 시간
- 사용자 선호도 (알림 카테고리 등)
- 브라우저/기기 정보

```
// 서버로 전송되는 데이터 예시
{
  "userId": "user_123",
  "subscription": {
    "endpoint": "https://fcm.googleapis.com/fcm/send/dxVEn-ACR8k:APA91bE...",
    "expirationTime": null,
  }
}
```

```
    "keys": {
      "p256dh": "BLGMvz2kBSL4riRWfC4XboJD-Dn9r7aZ9TdmhwA...",
      "auth": "tBHitJI4PYR9AOjDDk47Cw=="
    },
    "createdAt": "2024-05-04T10:30:00Z",
    "userAgent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36..."
  }
}
```

서버는 이 구독 정보를 데이터베이스에 저장하고, 나중에 이 정보를 사용하여 특정 사용자 또는 기기에 푸시 알림을 전송합니다.

5. 푸시 메시지 전송 및 처리 흐름



JWT 토큰 구조

```
{
  "header": { "typ": "JWT", "alg": "ES256" },
  "payload": {
    "aud": "https://fcm.googleapis.com", // 푸시 서비스 도메인
    "exp": 1710000000, // 만료 시각
    "sub": "mailto:관리자@도메인.com" // 발신자 이메일
  },
  "signature": "..." // ECDSA 서명
}
```

주요 단계

1. **서버 측 준비**: JWT 토큰 생성 및 메시지 암호화
2. **HTTP 요청**: 푸시 서비스에 암호화된 메시지 전송
3. **인증 검증**: 푸시 서비스가 JWT 서명과 페이로드 유효성 확인
4. **메시지 전달**: 브라우저로 암호화된 메시지 전송
5. **Service Worker 활성화**: 메시지 수신 시 깨어남
6. **알림 표시**: 암호화된 메시지 복호화 후 사용자에게 알림 표시

6. 알림 클릭 처리 및 사용자 상호작용

```
// Service Worker에서 알림 클릭 처리
self.addEventListener('notificationclick', event => {
  // 알림 닫기
  event.notification.close();

  // 관련 URL 열기
  if (event.notification.data?.url) {
    event.waitUntil(
      clients.openWindow(event.notification.data.url)
    );
  }
});
```

처리 흐름

1. 사용자가 알림 클릭
2. 'notificationclick' 이벤트 발생
3. 알림 데이터에서 URL 추출
4. 해당 URL로 새 창 열기 또는 기존 창 포커스

7. 구독 데이터 구조

A. PushSubscription 객체 구조

```
{
  "endpoint": "https://fcm.googleapis.com/fcm/send/c5pTy...",
  "expirationTime": null,
  "keys": {
    "p256dh": "BNcRdreAL..", // ECDH P-256 공개키
    "auth": "tBHItJI5..." // 인증 토큰
  }
}
```

B. 서버 측 저장 구조

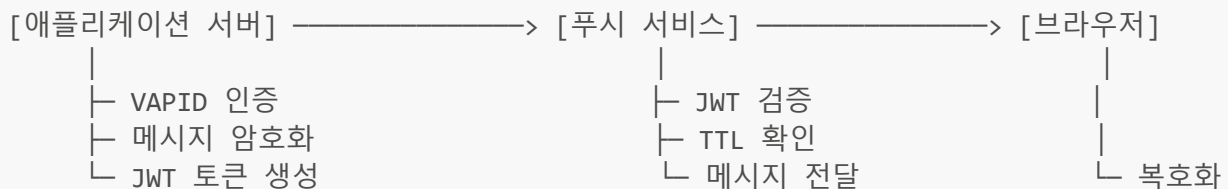
```
{
  "user_id": "사용자식별자",
  "subscription": {
    "endpoint": "https://fcm.googleapis.com/...",
    "keys": {
      "p256dh": "BNcRdreAL..",
      "auth": "tBHItJI5..."
    }
  },
  "created_at": "2024-01-01T00:00:00Z"
}
```

8. 보안 메커니즘

A. 핵심 보안 계층

1. **서버 인증 (VAPID)**: JWT + ECDSA 서명으로 발신자 신원 증명
2. **메시지 보안**: p256dh 키 기반 엔드-투-엔드 암호화
3. **접근 제어**: Endpoint URL 기반 타겟팅, TTL 기반 메시지 수명 관리

B. 보안 플로우



9. 전체 프로세스 요약

1. **초기 설정**: VAPID 키 생성, Service Worker 등록
2. **구독 설정**: 사용자 권한 획득, 푸시 서비스 구독, 서버에 정보 저장
3. **메시지 전송**: 서버에서 JWT 생성 및 메시지 암호화, 푸시 서비스로 전송
4. **알림 표시**: Service Worker에서 메시지 수신, 복호화, 알림 표시

5. 사용자 상호작용: 알림 클릭 시 관련 페이지 열기

보안 특징

- 엔드-투-엔드 암호화로 메시지 내용 보호
- 다중 인증 레이어로 무단 메시지 방지
- 사용자 권한 기반 접근 제어

VAPID를 위한 JWT 토큰 생성 과정

VAPID JWT 토큰은 푸시 서비스가 메시지 발신자의 신원을 확인하기 위해 사용됩니다. 다음은 생성 과정입니다:

1. VAPID 키 준비

우선 ECDSA(타원 곡선 디지털 서명 알고리즘) 키 쌍이 필요합니다:

```
// 서버에서 VAPID 키 생성 (한 번만 수행하고 안전하게 저장). 외부 툴을 사용하여 VAPID 키  
// 생성하여 웹 서버에 저장하는 방법도 있다.  
const webpush = require('web-push');  
const vapidKeys = webpush.generateVAPIDKeys();  
  
console.log('Public Key:', vapidKeys.publicKey); // 클라이언트에 전달할 키  
console.log('Private Key:', vapidKeys.privateKey); // 서버에만 저장할 비밀 키
```

2. JWT 헤더 및 페이로드 생성

```
// 1. JWT 헤더 생성  
const header = {  
  typ: "JWT", // 토큰 타입  
  alg: "ES256" // 서명 알고리즘 (ECDSA + SHA-256)  
};  
  
// 2. 페이로드 생성  
const payload = {  
  aud: "https://fcm.googleapis.com", // 대상 푸시 서비스 URL  
  exp: Math.floor(Date.now() / 1000) + 12 * 3600, // 만료 시간 (12시간)  
  sub: "mailto:관리자@도메인.com" // 발신자 연락처  
};
```

3. 서명 생성

```
// 3. 헤더와 페이로드를 base64url로 인코딩  
const encodedHeader = base64UrlEncode(JSON.stringify(header));  
const encodedPayload = base64UrlEncode(JSON.stringify(payload));  
  
// 4. 서명할 데이터 준비
```

```
const signatureInput = `${encodedHeader}.${encodedPayload}`;

// 5. 비공개 키로 서명 생성
// ECDSA-P256 알고리즘으로 서명 (VAPID 비공개 키 사용)
const signature = createECDSASignature(signatureInput, vapidKeys.privateKey);
const encodedSignature = base64UrlEncode(signature);

// 6. 최종 JWT 토큰 생성
const jwtToken = `${encodedHeader}.${encodedPayload}.${encodedSignature}`;
```

4. 푸시 메시지 전송에 JWT 활용

```
// 7. 웹 푸시 메시지 전송 시 인증 헤더로 사용
const options = {
  vapidDetails: {
    subject: "mailto:관리자@도메인.com",
    publicKey: vapidKeys.publicKey,
    privateKey: vapidKeys.privateKey
  },
  TTL: 60 * 60 // 1시간
};

// 웹 푸시 라이브러리는 내부적으로 JWT 토큰을 생성하여 Authorization 헤더에 포함
await webpush.sendNotification(
  pushSubscription, // 사용자의 구독 정보
  JSON.stringify({title: "알림 제목", body: "알림 내용"}),
  options
);
```

키의 역할

1. 비공개 키(Private Key):

- JWT 토큰의 서명 생성에만 사용
- 서버에 안전하게 보관되어야 함
- ECDSA 알고리즘으로 토큰에 서명하는 데 활용

2. 공개 키(Public Key):

- 구독 과정에서 클라이언트에 전달되어 `applicationServerKey`로 사용
- 푸시 서비스에서 JWT 서명 검증에 사용
- 누구나 볼 수 있어도 안전함

이 JWT 토큰은 애플리케이션 서버가 푸시 서비스에 자신을 인증하고, 푸시 메시지의 진정성을 보장하는 데 필수적입니다.

올바른 키 사용 방식에 대한 보충설명

```
const options = {
  vapidDetails: {
    subject: "mailto:관리자@도메인.com",
    publicKey: vapidKeys.publicKey, // 이것은 푸시 서비스에 전달됨
    privateKey: vapidKeys.privateKey // 이것은 서버에만 보관하고 절대 외부로 전송하
    지 않음
  },
  TTL: 60 * 60 // 1시간
};
```

위 코드는 서버 측에서 웹 푸시 메시지를 전송하기 위한 설정입니다. 이 코드에서:

1. **privateKey**:

- 서버에서만 사용되며 JWT 토큰을 서명하는 데 사용됩니다
- 푸시 서비스나 클라이언트로 전송되지 않습니다
- 웹푸시 라이브러리가 내부적으로 이 키를 사용해 JWT를 생성합니다

2. 키 사용 흐름:

- 서버는 **privateKey**로 JWT 토큰에 서명합니다
- JWT 토큰은 푸시 서비스에 전송됩니다 (privateKey 자체는 아님)
- 푸시 서비스는 **publicKey**를 사용해 JWT 서명을 검증합니다

중요 개념: 이 과정에서 개인 키 자체가 푸시 서비스로 전송되는 것이 아니라, 개인 키로 서명된 JWT 토큰이 전송되는 것입니다. 이것이 공개키 암호화의 기본 원리입니다.

web-push 라이브러리를 사용할 때 **sendNotification** 함수는 내부적으로 JWT를 생성하고 Authorization 헤더에 포함시켜 푸시 서비스에 요청을 보냅니다.

VAPID 키와 웹푸시 전송 메커니즘 보충설명

options에 **privateKey**가 포함되어 있다고 해서 그 키 자체가 푸시 서비스에 전송되는 것은 아닙니다.

webpush.sendNotification 내부 동작 방식

```
const options = {
  vapidDetails: {
    subject: "mailto:관리자@도메인.com",
    publicKey: vapidKeys.publicKey,
    privateKey: vapidKeys.privateKey // 이 키는 전송되지 않음
  },
  TTL: 60 * 60 // 1시간
};

await webpush.sendNotification(
  pushSubscription,
  JSON.stringify({title: "알림 제목", body: "알림 내용"}),
```

```
options
);
```

web-push 라이브러리의 sendNotification 함수 동작 방식:

1. 키 사용 방법:

- **privateKey**는 서버 내부에서만 사용되어 JWT 토큰에 서명하는 데 사용됩니다
- 라이브러리가 내부적으로 이 키를 사용해 JWT 토큰을 생성합니다
- 생성된 JWT 토큰만 푸시 서비스로 전송됩니다 (키 자체는 전송되지 않음)

2. 실제 요청에 포함되는 내용:

```
POST [endpoint URL]
Headers:
- Authorization: vapid t=[JWT 토큰]
- TTL: 3600
- Content-Type: application/octet-stream
- Content-Encoding: aes128gcm
Body: [암호화된 메시지 페이로드]
```

3. privateKey 활용 과정:

- 라이브러리가 **privateKey**로 JWT 토큰에 서명합니다
- 서명된 JWT 토큰은 **Authorization** 헤더에 포함됩니다
- 원본 **privateKey**는 절대 전송되지 않습니다

web-push 라이브러리는 이러한 복잡한 과정을 추상화하여 개발자가 간단히 사용할 수 있게 만들었습니다. 개발자는 키를 제공하기만 하면 라이브러리가 내부적으로 적절한 인증 및 암호화 처리를 수행합니다.

결론적으로, **options** 객체에 **privateKey**가 포함되어 있지만, 이것은 JWT 서명을 위해 로컬에서만 사용되며 실제 네트워크 요청에는 포함되지 않습니다.

웹 푸시 메시지 준비 상세 과정

1. 필요한 키의 종류와 생성 시점

웹 푸시 알림 시스템에서는 두 가지 주요 키 쌍이 사용됩니다:

1.1. VAPID 키 쌍 (서버 측)

- **생성 시점:** 서비스 초기 설정 시 한 번만 생성
- **용도:** 푸시 서비스가 푸시 메시지 발신자의 신원을 확인
- **보관 위치:** 서버 환경 변수나 안전한 저장소

```
// 서버에서 VAPID 키 생성
const webpush = require('web-push');
```

```
const vapidKeys = webpush.generateVAPIDKeys();

console.log('Public Key:', vapidKeys.publicKey); // 클라이언트에 전달할 키
console.log('Private Key:', vapidKeys.privateKey); // 서버에만 안전하게 보관
```

1.2. 사용자 구독 키 쌍 (클라이언트 측)

- **생성 시점:** 사용자가 푸시 알림을 구독할 때 브라우저가 자동 생성
- **용도:** 푸시 메시지 암호화 및 사용자 식별
- **키 종류:**
 - **p256dh:** 메시지 암호화에 사용되는 공개 키
 - **auth:** 인증 비밀 - 메시지 암호화 및 인증에 사용

```
// 클라이언트에서 생성된 구독 객체 예시
{
  "endpoint": "https://fcm.googleapis.com/fcm/send/dxVEn-ACR8k:APA91bE...",
  "expirationTime": null,
  "keys": {
    "p256dh": "BLGMvz2kBSL4riRwfc4XboJD-Dn9r7aZ9TdmhWA...",
    "auth": "tBHitJI4PYR9AOjDDk47Cw=="
  }
}
```

2. JWT 토큰 생성 과정

2.1. JWT 구조

```
{
  "header": { "typ": "JWT", "alg": "ES256" },
  "payload": {
    "aud": "https://fcm.googleapis.com", // 푸시 서비스 URL
    "exp": 1710000000, // 만료 시각 (현재 시간 + 12시간)
    "sub": "mailto:admin@example.com" // 발신자 이메일
  },
  "signature": "..." // ECDSA 서명
}
```

2.2. 상세 생성 과정

```
const webpush = require('web-push');
const crypto = require('crypto');

// 1. JWT 헤더 생성 및 인코딩
const header = {
  typ: "JWT",
  alg: "ES256" // ECDSA + SHA-256
```

```

};
const encodedHeader = base64UrlEncode(JSON.stringify(header));

// 2. JWT 페이로드 생성 및 인코딩
const payload = {
  aud: "https://fcm.googleapis.com", // 대상 푸시 서비스
  exp: Math.floor(Date.now() / 1000) + 43200, // 현재 시간 + 12시간(초)
  sub: "mailto:admin@example.com" // 발신자 이메일
};
const encodedPayload = base64UrlEncode(JSON.stringify(payload));

// 3. 서명할 데이터 준비
const signatureInput = `${encodedHeader}.${encodedPayload}`;

// 4. VAPID 비공개 키로 서명 생성
const signature = crypto.createSign('sha256')
  .update(signatureInput)
  .sign({
    key: vapidKeys.privateKey,
    dsaEncoding: 'ieee-p1363' // ECDSA 서명 형식
  });
const encodedSignature = base64UrlEncode(signature);

// 5. 최종 JWT 토큰 조합
const jwtToken = `${encodedHeader}.${encodedPayload}.${encodedSignature}`;

// Base64 URL 인코딩 함수
function base64UrlEncode(str) {
  return Buffer.from(str)
    .toString('base64')
    .replace(/\+/g, '-')
    .replace(/\//g, '_')
    .replace(=/\//g, '');
}

```

3. 메시지 암호화 과정

웹 푸시 메시지는 ECDH(타원 곡선 디피-헬만) 키 교환과 AES-GCM을 사용하여 암호화됩니다.

3.1. 암호화 절차

```

// 1. 임시 ECDH 키 쌍 생성
const localKeyPair = crypto.createECDH('prime256v1');
localKeyPair.generateKeys();

// 2. 사용자 구독의 p256dh 키를 사용하여 공유 비밀(shared secret) 생성
const userPublicKey = Buffer.from(subscription.keys.p256dh, 'base64');
const sharedSecret = localKeyPair.computeSecret(userPublicKey);

// 3. 정보 혼합(Info) 생성
const info = createInfo('aesgcm', userPublicKey, localKeyPair.getPublicKey());

```

```
// 4. HKDF를 사용하여 암호화 키와 nonce 생성
const hkdf = new HKDF('sha256', subscription.keys.auth, sharedSecret, info);
const { key, nonce } = hkdf.derive(32, 12);

// 5. AES-GCM으로 메시지 암호화
const cipher = crypto.createCipheriv(
  'aes-128-gcm',
  key.slice(0, 16), // AES-128은 16바이트 키 사용
  nonce
);

// 6. 메시지 암호화
let encryptedMessage = cipher.update(JSON.stringify(messagePayload), 'utf8');
encryptedMessage = Buffer.concat([encryptedMessage, cipher.final()]);

// 7. 암호화된 메시지에 인증 태그 추가
const authTag = cipher.getAuthTag();
const encryptedPayload = Buffer.concat([encryptedMessage, authTag]);
```

4. 전체 푸시 메시지 생성 예시

4.1. 서버 측 전체 코드 예시

```
const webpush = require('web-push');
const crypto = require('crypto');

// VAPID 키 설정
const vapidKeys = {
  publicKey: 'BLGMvz2kBSL4riRWfC4XboJD-Dn9r7aZ9TdmhWA_iQJHIQNbxPzyX...',
  privateKey: 'KwEvmhQr3RN9pGFDU2XjQi7vi6vYJ1uNipSNxVs5RGM'
};

// 웹푸시 설정
webpush.setVapidDetails(
  'mailto:admin@example.com',
  vapidKeys.publicKey,
  vapidKeys.privateKey
);

// 사용자 구독 정보
const subscription = {
  endpoint: 'https://fcm.googleapis.com/fcm/send/dxVEn-ACR8k:APA91bE...',
  keys: {
    p256dh: 'BLGMvz2kBSL4riRWfC4XboJD-Dn9r7aZ9TdmhWA...',
    auth: 'tBHitJI4PYR9A0jDDk47Cw=='
  }
};

// 메시지 페이로드
const payload = JSON.stringify({
```

```

    title: '새 메시지가 도착했습니다',
    body: '홍길동님이 메시지를 보냈습니다: 안녕하세요, 물건 구매 가능할까요?',
    icon: '/icons/notification-icon.png',
    badge: '/icons/badge-icon.png',
    data: {
      url: '/chats/123',
      chatId: 123,
      senderId: 456
    },
    actions: [
      { action: 'reply', title: '답장' },
      { action: 'close', title: '닫기' }
    ]
  });

// 푸시 옵션
const options = {
  TTL: 60 * 60, // 1시간
  vapidDetails: {
    subject: 'mailto:admin@example.com',
    publicKey: vapidKeys.publicKey,
    privateKey: vapidKeys.privateKey
  },
  urgency: 'high',
  topic: 'chat-message'
};

// 푸시 메시지 전송
webpush.sendNotification(subscription, payload, options)
  .then(response => {
    console.log('푸시 알림 전송 성공:', response.statusCode);
  })
  .catch(error => {
    console.error('푸시 알림 전송 실패:', error);
  });

```

4.2. 실제 네트워크 요청 형태

```

POST https://fcm.googleapis.com/fcm/send/dxVEn-ACR8k:APA91bE...
Content-Type: application/octet-stream
Content-Encoding: aes128gcm
Content-Length: 280
TTL: 3600
Authorization: vapid
t=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJodHRwczovL2ZjbS5nb29nbGVhcGlzLmNvbSIsImV4cCI6MTcxMDAwMDAwMCwic3ViIjoibWFpbHRvOmFkbWluQGV4YW1wbGUuY29tIn0.9WdGiS0kGsh2nU-G_Vkwct9eAYEoAkW9SgcMnKJv5h1TLiLR3RpzHXJvp4mF6xPHgVciQJF4Jb58aR0JSrgA_w
Urgency: high
Topic: chat-message

[암호화된 바이너리 데이터]

```

5. 키 사용 흐름 요약

1. VAPID 키:

- 서버에서 생성 → 공개 키는 클라이언트에 제공 → 비공개 키는 서버에만 저장
- JWT 토큰 서명에 비공개 키 사용 → 푸시 서비스는 공개 키로 검증

2. 구독 키:

- 브라우저에서 자동 생성 → 서버에 전송 → 서버 데이터베이스에 저장
- 메시지 암호화에 사용 → 각 사용자마다 고유한 키

3. 암호화 키:

- 메시지 전송 시 임시 생성 → 사용자 구독 키와 결합 → 메시지 암호화
- 푸시 서비스로 전송 → 브라우저만 복호화 가능

이 모든 과정은 **web-push** 라이브러리가 내부적으로 처리하므로, 개발자는 구독 정보와 메시지 내용만 제공하면 됩니다.

Similar code found with 1 license type

참고 자료

- 웹 푸시 알림의 동작 원리와 보안 구조 (geundung.dev)